# Quality for fitnessfirst.de

Find the expensive mistakes first, and find them early.

# My starting point

### HOW I PREPARED

- Explored the website as a visitor to understand the flows
- Checked if it feels slow as a user, or unusually slow (PageSpeed Insights, mobile)
- Looked at whether the site is responsive and ready for mobile
- Looked at accessibility (automatic scores only, no full audit)
- Nothing submitted, no login, no tests on your live site

### WHAT I ASSUMED

- You have test environments next to production
- Payment providers are third parties, so they cannot be tested on production
- Most users use a phone, not a laptop

# How I explore

Exploratory testing: short sessions with a clear goal.

**Example goal** "Explore the checkout by changing my mind often, to find wrong totals."

### WHAT I WRITE DOWN EVERY TIME

- Goal and setup (device, browser, page version)
- What I covered
- Bugs, with steps and screenshots
- Open questions
- What I did NOT test

Saved next to the ticket, so the team can see it.

# What I learned: the areas that matter most

**Risk = how bad if it breaks x how likely it breaks**

1. Membership checkout
2. Cookie consent and tracking
3. Pause, cancel, withdrawal
4. Trial training booking
5. Campaigns and offers
6. Club search
7. Member login
8. Course plan
9. Magazine, career

# My plan for each risk

| Area | Main risk | My plan |
|---|---|---|
| **Membership checkout** | Wrong price, or a contract that never arrives | Price rule tests, end-to-end tests, a check in production every 15 minutes |
| **Cookie consent and tracking** | Marketing trackers load after "decline" | Automatic check after "decline all", review with the data protection officer |
| **Pause, cancel, withdrawal** | A legal flow stops working | End-to-end test on every release, texts checked by hand |
| **Trial training booking** | Slot does not exist, booking never reaches the club | One end-to-end booking per phone type, check that it arrives |
| **Campaigns and offers** | An old offer still shows after it ended | Date tests, look at what the cache shows |

**The other areas** lighter checks, mostly by hand.

# Flow 1: Membership checkout, the risks

5 STEPS

Club → Plan → Extras → Your details → Payment

## WHAT CAN GO WRONG

- Wrong price: club, plan, campaign, age, country
- Summary does not match the choice
- Customer gets stuck and leaves
- Contract or payment does not reach your systems
- Legal text missing or wrong

**Most expensive** a wrong price or a contract that was never created.

# Flow 2: Trial training booking

3 STEPS

Club → Date and time → Your details

### WHAT CAN GO WRONG

- Booked slot does not exist at the club
- Booking never reaches the club
- Date mistakes: today, weekend, month change, holidays
- Slow to react: 361 ms per tap for real users (good: 200 ms)

**Automate** slot rules, booking arrives at the club, one end-to-end booking per phone type.

# Devices, users, outside systems

Devices: desktop browser first, then the mobile browser view. Real phones: iPhone Safari, Android Chrome.

**Screen widths** 360, 390, 768, 1280 pixels.

**Users** new visitors, students, people switching gyms, visitors from Austria, keyboard and screen reader users, slow mobile internet, people who decline cookies.

### OUTSIDE SYSTEMS I SAW

- Usercentrics (cookie consent)
- VWO (A/B tests: not everyone sees the same page)
- Google Tag Manager and about 15 ad services
- Probably Baqend and Cloudflare (speed and caching)
- Unknown to me: payment, booking, member system

# Where I start: end-to-end tests by priority

END-TO-END TESTS, BUILT IN ORDER OF PRIORITY

1. **Priority 1:** membership checkout, trial booking, pause and cancel, cookie consent check. Built first in a desktop browser.
2. **Same tests, mobile view:** the priority 1 tests again in the mobile browser view, on a phone-sized screen.
3. **Priority 2:** campaigns and offers, club search, member login.
4. **The rest:** lighter checks, mostly by hand.

# Flow 1: what I would automate

Test pyramid: many small fast tests at the bottom, few slow ones at the top.

**Exploratory (manual):** new campaigns, odd combinations, Austria

**End-to-end (few):** buy a plan up to the payment step, on iPhone Safari and Android Chrome. Change the club mid-way: summary updates.

**Visual and accessibility:** price cards and summary look right, keyboard works, errors are read out

**API and integration:** the right plans and prices per club. Contract reaches the member system. Payment provider in test mode.

**Unit (many):** every price rule. Campaign start and end dates.

**Plus in production** a robot check of steps 1 to 3 every 15 minutes.

# What I automate, and what I don't

### AUTOMATE WHEN

- it runs often, matters a lot, and the expected result is clear
- it is stable and cheap to maintain
- Example: price rules (unit tests)

### KEEP MANUAL WHEN

- it is new or still changing
- it needs human judgement: look, wording, screen reader
- Examples: a new campaign page, the screen reader on checkout

### WHAT I WOULD NOT TEST

- Outside services themselves: payment provider, cookie tool, ad services. I test only our side.
- Load and security attacks on the live site
- Low-risk areas in depth, like the magazine: a quick look is enough

# The quality process I would encourage

Six phases, one question: what could go wrong, and how do we prevent it?

Bugs cost less when we find them early.

1
**Requirements**
understand the why

2
**Risk**
decide together what matters most

3
**Development**
explore and pair early

4
**Automation**
automate the right things

5
**Release**
done is a criterion, not a feeling

6
**After release**
learn from what got through

# Quality is the team's job

| | |
|---|---|
| **Product** | clear goals and acceptance criteria. Decides on risk together with the team. |
| **Developers** | unit and API tests for their code. Fix what breaks the pipeline. |
| **Business departments** | check that it is right for the business (acceptance). |
| **QA** | gate keeper at release, and a resource for everyone:<br>• helps write acceptance criteria<br>• pairs with developers<br>• keeps the test cases alive<br>• coaches the team on testing |

# From idea to live website

| Feature | Merge request | Dev | Stage | Production | After release |
| --- | --- | --- | --- | --- | --- |
| acceptance criteria and risks agreed | code review, unit tests, code checks | API and integration tests, smoke test | end-to-end regression, visual, accessibility, page speed, consent check, acceptance by business | smoke test right after deploy | monitoring and real-user page speed. Rollback ready. |

**Test data** no real customer data on Stage. Copied data is anonymised.

# When do we stop a release?

### STOPS THE RELEASE

- Wrong price, checkout or booking broken
- Cancel or pause broken
- Cookie decline does not work
- Security problem, or someone sees another person's data
- Main flow not usable with a keyboard

### CAN GO, WITH CONDITIONS

- Small bug with an easy workaround, ticket and fix date
- Problem on a low-traffic page
- New feature switched off with a feature flag

**Unreliable tests** run once more. If it passes, it goes to quarantine with an owner and a deadline. Never just ignored.

# Beyond "does it work"

| Area | Automated | Manual | Limit (gate) |
|---|---|---|---|
| **Accessibility** | Accessibility scan on every merge request | Keyboard and screen reader on main flows every release, expert audit once a year | No critical issues on checkout and booking |
| **Data protection and consent** | After "decline all", only allowed services load | Check banner texts with the data protection officer | Any marketing tracker after decline stops the release |
| **Page speed** | Lighthouse in the pipeline, timing of booking steps | Look at real-user data every week | Real users: LCP 2.5 s, INP 200 ms, CLS 0.1. Pipeline: no page slower than today |

**Today, real users, mobile** all 3 pages load fast, all 3 react too slowly.

Membership INP 227 ms, trial training 361 ms.

# Where AI helps, and where it doesn't

### AI HELPS WITH

1. **Requirements:** finds gaps and missing edge cases in acceptance criteria
2. **Exploratory testing:** agents explore with a goal and must show evidence for every result
3. **Failed tests:** groups failures, suggests "real bug" or "flaky"

### ALWAYS WITH PEOPLE

- Final priority of bugs
- Go / no-go
- Legal, accessibility and data protection decisions

**Rules** no real customer data in AI tools. Every result needs evidence. AI must list what it did not test.



# Extra slides

# Page speed details

| Page (mobile) | Real users: LCP | Real users: INP | Real users: CLS | Lab score |
|---|---|---|---|---|
| **Homepage** | 2 s | 208 ms | 0.03 | 43 |
| **Membership** | 1.1 s | 227 ms | 0.04 | 36 |
| **Trial training** | 1.3 s | 361 ms | 0.05 | 69 |

Limits for good (75% of visits): LCP 2.5 s, INP 200 ms, CLS 0.1. Source: PageSpeed Insights, 27 October 2026. Lab numbers come from single runs and vary.

# Test data and environments

- Stage: anonymised copy of production, or generated test data
- Fixed test accounts and test clubs, reset regularly
- Payment provider in test mode, never real payments
- Outside systems we cannot control: simulated (mocked) in API tests, real only in a few end-to-end tests on Stage
- Test emails only on reserved test domains

# My AI testing agents (designed, not yet run on a live site)

- 4 agents: explorer, component, flow, API
- Rules: only allowed test sites, no real purchases, secrets hidden, evidence for every result
- Results: pass, fail with priority, or "suspicious" with a reason
- Several runs are merged, because AI results vary from run to run
- A P1 failure can block the pipeline. Humans decide on the rest.

# My first weeks

| | |
|---|---|
| **Weeks 1 to 2** | learn the product, the team and the release process. Read the bug history. |
| **Weeks 3 to 4** | risk map with the team, first quick wins (consent check, smoke tests) |
| **Month 2 to 3** | regression suite for checkout and booking, quality gates in the pipeline |

# One normal week

| | |
|---|---|
| **Planning** | Three Amigos on new stories, risks agreed |
| **Every day** | pairing, merge request checks, quick questions |
| **During the sprint** | exploratory sessions on new features |
| **Before release** | regression on Stage, acceptance with business |
| **Release day** | smoke test, monitoring |
| **After release** | look at what got through, add a test for it |